

東京 2025
LINUX PLUMBERS CONFERENCE

TOKYO, JAPAN / DECEMBER 11-13, 2025

Linux Networking with MANA: RX Path Optimization and Netshaper Bandwidth Control

Presenters: Dipayaan Roy, Erni Sri Satya Vennela.





Part I: MANA RX Path Optimization.





What is MANA?

- **MANA = Microsoft Azure Network Adapter**
- Microsoft's in-house SmartNIC for high performance Azure compute.
- SR-IOV based, exposed as VF(Virtual function) to Linux guest & as PF to DOM0 Host, providing strong isolation.
- Designed for high throughput: **up to 200 Gbps**.
- Support Hardware offloads capabilities: TSO, checksum, etc. (no support for LRO yet)
- Driver is up streamed since April 2021 (drivers/net/ethernet/microsoft/mana)

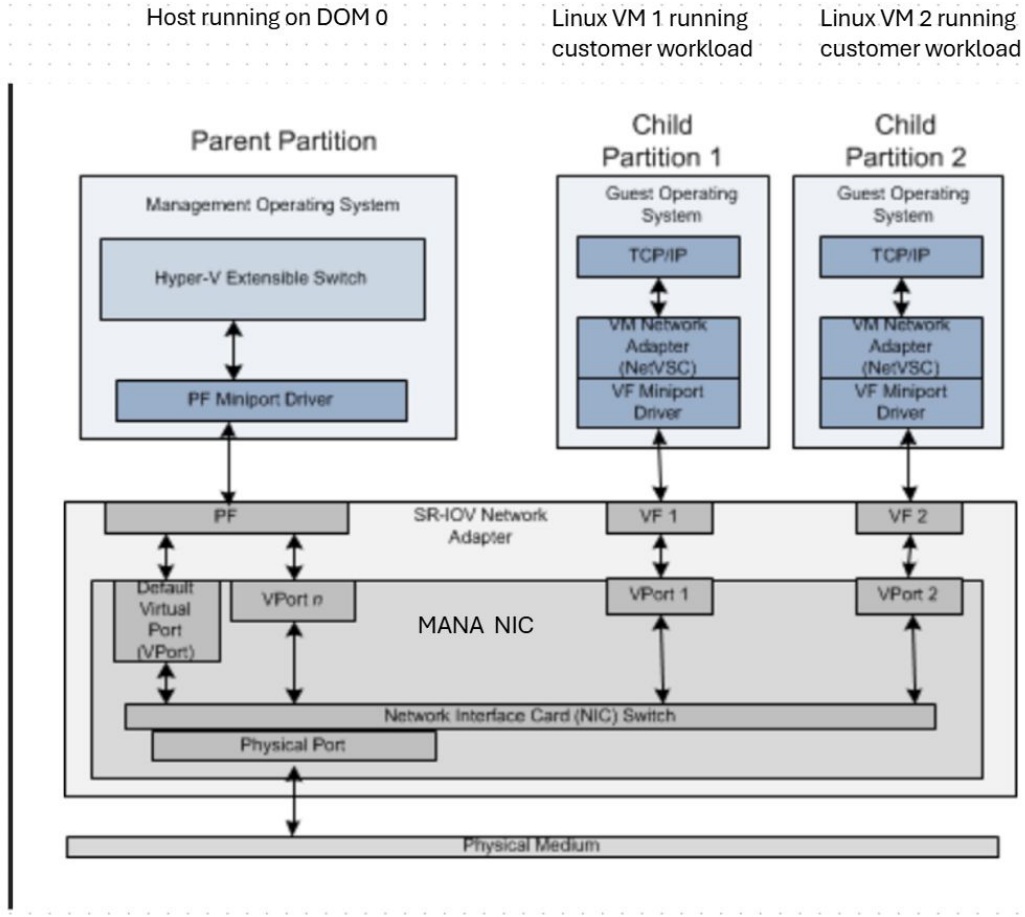
Motivation: Why We Had to Touch RX path

- Azure VMs: historically x86-64, 4 KB base page
- New ARM64 SKUs (e.g., ND GB200-v6, Cobalt 100,200) → configurable base page sizes (4 KB, 16 KB, 64 KB)
- Same driver, same kernel → very different behaviour on 64 KB
- Symptom: Very Low throughput than expected on 64 KB systems and Higher memory footprint,
- Initial bring up gives roughly ~60 Gbps with 48 threads, for same driver and hardware delivers ~200 Gbps on other architectures.

```
pse@pse-2404-latest:~/ntttcp-for-linux/src$ uname -a
Linux pse-2404-latest 6.8.0-1019-azure-nvidia #20-Ubuntu SMP Fri Jun 27 23:52:00 UTC 2025 aarch64 aarch64 aarch64 GNU/Li
nux
pse@pse-2404-latest:~/ntttcp-for-linux/src$ getconf PAGE_SIZE
65536
pse@pse-2404-latest:~/ntttcp-for-linux/src$ sudo ./ntttcp -s -m 48,*,192.168.0.73 -b 1000k -p 8010 -x test.xml -W 10 -t
60
NTTTCP for Linux 1.4.0
---------------------------------------------------------
15:40:13 INFO: Test cycle time negotiated is: 70 seconds
15:40:13 INFO: 48 threads created
15:40:13 INFO: 48 connections created in 3188 microseconds
15:40:13 INFO: Network activity progressing...
15:40:23 INFO: Test warmup completed.
Real-time throughput: 56.71Gbps
```

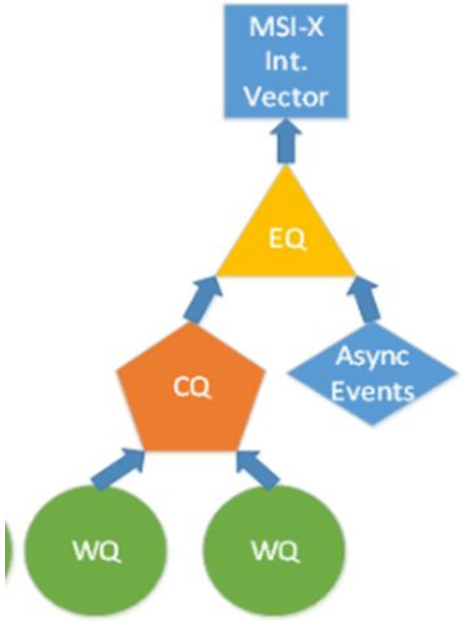
MANA Architecture and Configuration in Azure.

- MANA NIC sits at the bottom of the Azure virtualization stack and connects to the physical network medium.
- The NIC exposes multiple virtual ports (VPorts).
- The Physical Function (PF) is owned by the parent partition (host/Dom0).
- Each Azure VM runs as a child partition with its own guest OS.
- Each VM is directly assigned a Virtual Function (VF), mapped to a NIC VPort.
- The VF bypasses the parent's networking stack and communicates directly through the hyperv-PCIe interface.



MANA configuration in AZURE

MANA Queue-Based Architecture: WQ, CQs, EQs, and MSI-X.



- Each VF has Work Queues (WQ), Completion Queues (CQ), and Event Queues (EQ).
- Driver populates TX/RX WQ descriptors that the NIC consumes to send or receive packets.
- Each TX and RX WQ has an associated CQ.
- RX Path:
  - Driver posts receive buffers into the RX WQ.
  - NIC places incoming packets into these buffers.
  - For each completed packet, NIC writes an CQE into the corresponding CQ.
- TX Path:
  - Driver posts send descriptors into the TX WQ and rings the doorbell.
  - After transmitting packets, NIC writes TX CQEs into the CQ.
- Event Queue
  - Multiple CQs are grouped under an Event Queue (EQ).
  - When any CQ has new completions, NIC writes an EQE to signal work pending.
  - Each EQ is tied to an MSI-X interrupt. On interrupt, driver acknowledges it and schedules NAPI to poll CQs and process TX/RX completions.

RX Path in Network Drivers

Typical RX Flow:

Buffer Posting: Driver allocates pages from kernel, DMA-maps them, and posts as RX buffers in NIC's RX ring.

Packet Arrival: NIC performs **DMA write** into one of the posted buffers when a packet arrives.

NAPI Poll: NIC interrupt triggers NAPI poll → driver fetches the received page from RX ring.

SKB Construction: Driver wraps received data into **struct sk_buff** (SKB); page remains attached until SKB is freed.

Ring Refill: Once SKB is consumed, driver quickly **refills RX ring** with new buffers for next packets.

Efficiency Note:

Works well with **small base pages (4 KB)** where buffer size ≈ MTU (1500 B).

Becomes **highly wasteful with large pages (64 KB)** — one packet per page.

Current problem in the MANA Driver Rx path:

Current Design Issue

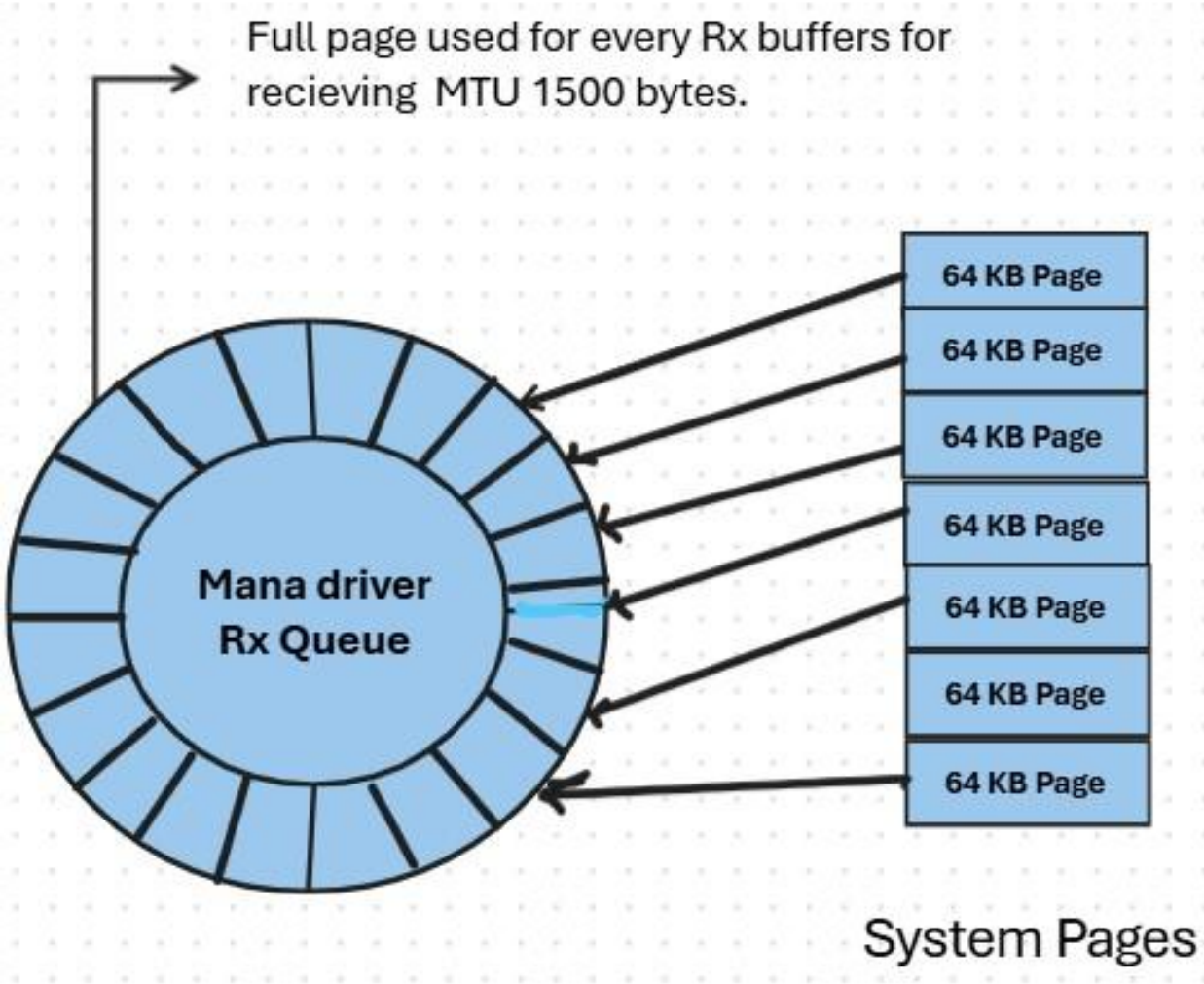
- Each RX buffer in MANA is allocated from a full **PAGE**, regardless of packet size (~1500 B MTU).
- Leads to severe inefficiency, especially on **ARM64 systems with 64 KB base pages**.

Key Problems

- **Memory wastage:** 1.5 KB packet occupies a full 64 KB page.
- **TLB & cache inefficiency:** Unused portions still consume TLB entries and pollutes CPU caches.
- **Throughput bottleneck:** High per-packet allocation overhead reduces packet-per-second performance.
- **High memory footprint:** Continuous page allocation under load causes frequent low-memory conditions.
- **Allocator & DMA overhead (x86-64 too):** Even with 4 KB pages, frequent page allocations and DMA mappings are costly.
- **single-TCP scenarios:** all packets map to one RX queue — per-packet overhead directly caps throughput.

Representation of the problem discussed

This figure makes the issue more visual. Each RX descriptor/buffer is backed by a full page, shown here as a 64 KB block. Only a small portion — about 1.5 KB — is actually used to hold packet data, and the rest remains idle. This pattern repeats across thousands of buffers, leading to massive memory waste and allocator overhead.



Solution:

The patch introduces two key optimizations into the MANA driver RX path:

1. page_pool Based Fragment Allocation

- Slice each large page into multiple **MTU-sized RX buffers** instead of 1 page per packet.
- On **64 KB pages**, up to **35 × 1500 B packets/page**, improving memory utilization significantly.
- **Improved cache locality:** Consecutive RX buffers from the same page, it enhance CPU cache reuse and reduces TLB misses significantly.
- **Reduced allocation churn:** Fewer costly pages allocation calls; faster buffer refills via fragment recycling.

2. Pre-DMA Mapped page_pool (per RX Queue)

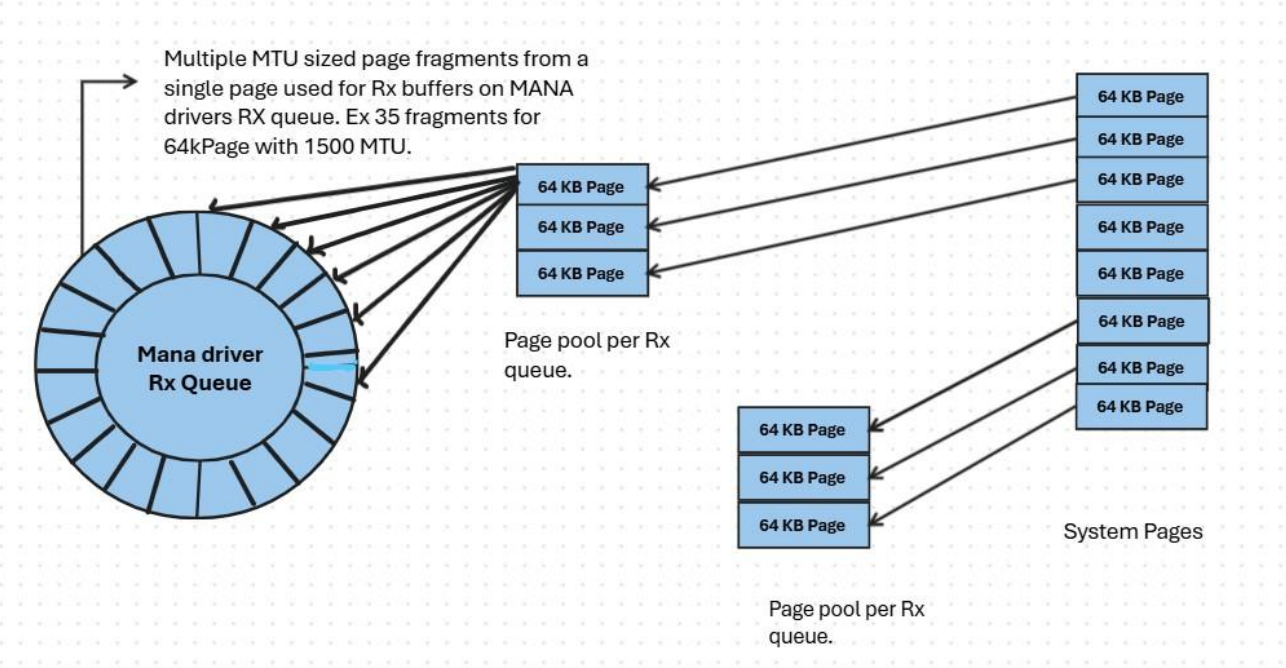
- Dedicated **page pool per RX queue**, created and DMA-mapped during queue initialization.
- **Persistent DMA mapping:** Pages remain mapped, eliminating repeated map/unmap operations.

The solution enables **faster buffer refills**, lower per-packet cost, reduced memory footprint, and **higher throughput**.

Representation of the solution discussed

Here's the same RX ring after the optimization. Each 64 KB page is now divided into multiple fragments, each used for one packet.

The page pool per RX queue maintains these pages in a pre-DMA mapped state. As packets are received and buffers recycled, the same page continues to serve new packets without costly re-mapping. This structure achieves both high memory reuse and improved throughput."



Performance Impact of the Patch on ARM64 systems (64KB pages, Standard MTU size of 1500):

Memory utilization Improvement

RX buffers per 64 KB page: **1 → up to 35**

Per-queue RX buffer usage: **512 MB → ~14.7 MB**

With **32 RX queues @ 8192 queue depth**:

- Total RX footprint reduced from : **16.4 GB → ~470 MB**
- **~97% reduction** in RX memory usage

Throughput Improvement

Single-queue / single TCP flow: **up to 200% improvement**

Multi-connection runs: **185+ Gbps**, very close to the **200 Gbps** card limit

Jumbo frames (9K MTU)

Up to **7 RX buffers per 64 KB page**

Significant gains in both **throughput** and **memory efficiency** here as well

Performance Impact of the Patch on x86-64 systems (4KB pages, Standard MTU size of 1500):

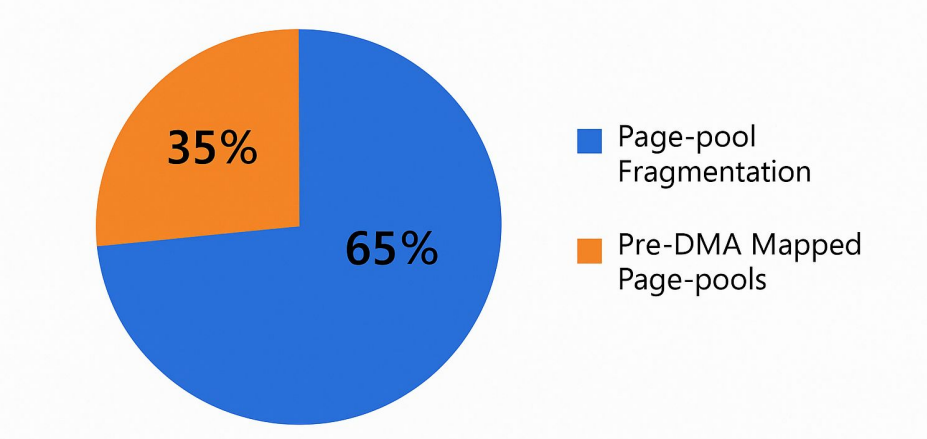
- **Memory Utilization Improvement:**
  Up to 2 RX buffers per 4KB page (vs. 1 before).

  Per-queue RX buffer usage reduced from **32 MB → 16 MB,**

  With 32 RX queues in MANA 8192 queue depth:
  Total RX footprint reduced from **1 GB → 512 MB,** Memory savings of ~ 50%.

- **Throughput Improvement in single queue/TCP flow:**
  Up by ~15% improvement.

Breaking Down the Performance Contributions



Based on experiments collected on GB200 ARM64 VMs (64 KB pages):

Total throughput gain: ~200 % in single-flow TCP

Page-pool Fragmentation:
→ Contributes **~65 to 70 %** of total improvement
→ Gains from drastic reduction in page allocation churn, improved cache locality, faster buffer refills by recycling fragments

Pre-DMA Mapped Page-Pools:
→ Contributes **~30 to 35 %** of total improvement
→ Gains from eliminating repeated DMA map/unmap overhead

Combined effect: Major boost in single-queue performance and sustained memory efficiency across all RX queues

From the patch: deciding the fragment count, size based on MTU & PAGE_SIZE.

```
/* Get RX buffer's data size, alloc size, XDP headroom based on MTU */
static void mana_get_rxbuf_cfg(struct mana_port_context *apc,
            int mtu, u32 *datasize, u32 *alloc_size,
            u32 *headroom, u32 *frag_count)
{
  u32 len, buf_size;

  /* Calculate datasize first (consistent across all cases) */
  *datasize = mtu + ETH_HLEN;

  /* For xdp and jumbo frames make sure only one packet fits per page */
  if (mtu + MANA_RXBUF_PAD > PAGE_SIZE / 2 || mana_xdp_get(apc)) {
    if (mana_xdp_get(apc)) {
      *headroom = XDP_PACKET_HEADROOM;
      *alloc_size = PAGE_SIZE;
    } else {
      *headroom = 0; /* no support for XDP */
      *alloc_size = SKB_DATA_ALIGN(mtu + MANA_RXBUF_PAD +
                  *headroom);
    }

    *frag_count = 1;
    return;
  }

  /* Standard MTU case - optimize for multiple packets per page */
  *headroom = 0;

  /* Calculate base buffer size needed */
  len = SKB_DATA_ALIGN(mtu + MANA_RXBUF_PAD + *headroom);
  buf_size = ALIGN(len, MANA_RX_FRAG_ALIGNMENT);

  /* Calculate how many packets can fit in a page */
  *frag_count = PAGE_SIZE / buf_size;
  *alloc_size = buf_size;
}
```

Patch status: accepted upstream!!

Accepted Upstream:
https://git.kernel.org/pub/scm/linux/kernel/git/netdev/net-next.git/commit/?id=730ff06d3f5c
Patch merged after multiple community reviews.

Kernel Inclusion:
Part of **Linux Kernel v6.18**,

Backports in Progress:
Being **backported by distro partners (e.g., Ubuntu)** based on customer request.

Multiple Azure SKUs Benefiting:
ND-**GB200 VMs**, ND-**GB200 Bare Metal**, Nv-**GB300**, **Ampere** based Sku offerings and In house Azure **Cobalt** platforms.

root@lisa--338-e0-n0: /home

```
root@lisa--338-e0-n0:/home/diproy# echo iperf client
iperf client
root@lisa--338-e0-n0:/home/diproy# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default q
len 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default
 qlen 1000
    link/ether 60:45:bd:e8:ff:d5 brd ff:ff:ff:ff:ff:ff
    inet 10.0.0.4/24 metric 100 brd 10.0.0.255 scope global eth0
       valid_lft forever preferred_lft forever
    inet6 fe80::6245:bdff:fee8:ffd5/64 scope link
       valid_lft forever preferred_lft forever
5: enP30832s1: <BROADCAST,MULTICAST,SLAVE,UP,LOWER_UP> mtu 1500 qdisc mq master eth
0 state UP group default qlen 1000
    link/ether 60:45:bd:e8:ff:d5 brd ff:ff:ff:ff:ff:ff
    altname enP30832p0s0
    inet6 fe80::6245:bdff:fee8:ffd5/64 scope link
       valid_lft forever preferred_lft forever
root@lisa--338-e0-n0:/home/diproy# iperf3 -c 10.0.0.5 -p 5201 -t 10
```

root@lisa--338-e0-n1: /home/

```
root@lisa--338-e0-n1:/home/diproy/net-next/net-next# echo GB200 Test VM running iperf server
GB200 Test VM running iperf server
root@lisa--338-e0-n1:/home/diproy/net-next/net-next# uname -a
Linux lisa--338-e0-n1 6.16.0-rc4+ #3 SMP Thu Jul 10 20:25:26 UTC 2025 aarch64 aarch64 aarch64 GNU/Linux
root@lisa--338-e0-n1:/home/diproy/net-next/net-next# lscpu | head -n 10
Architecture:                         aarch64
CPU op-mode(s):                       64-bit
Byte Order:                           Little Endian
CPU(s):                               128
On-line CPU(s) list:                  0-127
Vendor ID:                            ARM
BIOS Vendor ID:                       NVIDIA
Model name:                           Neoverse-V2
BIOS Model name:                      Grace A02P None CPU @ 0.0GHz
BIOS CPU family:                      258
root@lisa--338-e0-n1:/home/diproy/net-next/net-next# getconf PAGE_SIZE
65536
root@lisa--338-e0-n1:/home/diproy/net-next/net-next# lsmod | grep mana
mana                  262144  0
root@lisa--338-e0-n1:/home/diproy/net-next/net-next# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether 60:45:bd:e8:fb:aa brd ff:ff:ff:ff:ff:ff
    inet 10.0.0.5/24 metric 100 brd 10.0.0.255 scope global eth0
       valid_lft forever preferred_lft forever
    inet6 fe80::6245:bdff:fee8:fbaa/64 scope link
       valid_lft forever preferred_lft forever
14: enP30832s1: <BROADCAST,MULTICAST,SLAVE,UP,LOWER_UP> mtu 1500 qdisc mq master eth0 state UP group default qlen 1000
    link/ether 60:45:bd:e8:fb:aa brd ff:ff:ff:ff:ff:ff
    altname enP30832p0s0
    inet6 fe80::6245:bdff:fee8:fbaa/64 scope link
       valid_lft forever preferred_lft forever
root@lisa--338-e0-n1:/home/diproy/net-next/net-next#
```

Future work:

- **Should we adopt higher-order pages for page pools on x86, even with a 1500-byte MTU?**
- **Motivation:**
  - Aim to leverage gains similar to what we see with 64 KB base page sizes.
  - Reduce page allocation churn and improve cache locality.
- **Early Results (MANA Experiments):**
  - Page pools backed by higher-order pages (order-9 / 2 MB).25–30% additional improvement in single TCP connection throughput.
  - No observed negative impact on overall system performance.

Part II: Netshaper Bandwidth Control.



AGENDA

01 PROBLEM STATEMENT

02 PREVIOUS ATTEMPTS

03 OVERVIEW OF NETSHAPERS

04 IMPLEMENTATION

05 DEMO

06 CHALLENGES FACED

07 USERSPACE TOOL

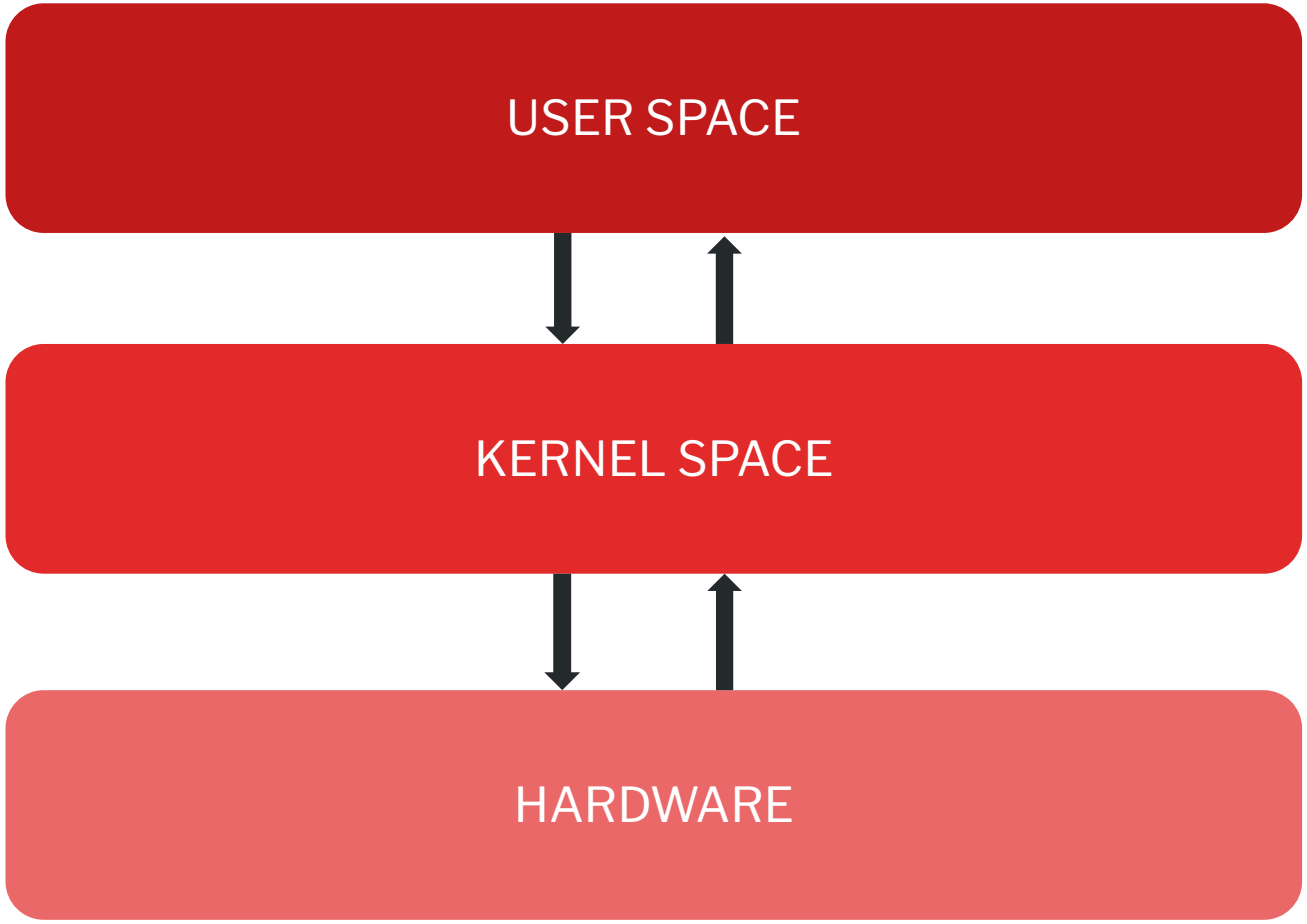
08 FUTURE SCOPE

PROBLEM STATEMENT

TRAINING IMPACTED BY VARYING LINK SPEEDS

THE NODE WITH HIGHER LINK SPEED SHOULD BE CLAMPED TO LOWER VALUE TO MATCH WITH OTHER NODE'S SPEED.

NEED: VMs TO REQUEST LOWER BANDWIDTH


USER SPACE
KERNEL SPACE
HARDWARE

Ethtool?

- Use “ethtool -s <interface> speed 1000”
- It will set the bandwidth internally and gives us the output:

```
$ethtool <interface>
Settings for <interface>:
        Supported ports: [  ]
        Supported link modes:   Not reported
        Supported pause frame use: No
        Supports auto-negotiation: No
        Supported FEC modes: Not reported
        Advertised link modes:  Not reported
        Advertised pause frame use: No
        Advertised auto-negotiation: No
        Advertised FEC modes: Not reported
        Speed: 1000Mb/s
        Duplex: Full
        Auto-negotiation: off
        Port: Other
        PHYAD: 0
        Transceiver: internal
        Link detected: yes
```

Drawback:

- It is important to report the Supported ports and Supported link modes
- Also, it is not the recommended way to clamp bandwidth

TC-HTB (HEIRARCHY TOKEN BUCKET)?

- HTB qdisc has hardware offload support.
- Create a class for a single node representing the whole network device:

Add the root qdisc

```
$ tc qdisc add dev <interface> root handle 1:
htb offload
```

Add the class with required rate

```
$ tc class add dev <interface> parent 1:
classid 1:1 htb rate 1000mbit
```

Display class details

```
$ tc class show dev <interface> classid 1:1
> class htb 1:1 root prio 0 rate 1Gbit ceil
1Gbit burst 1375b cburst 1375b  offload
```

Drawback:

- Using this method for creating only one node under utilizes its ability.
- HTB offload requires dynamic allocation of TX queues

NETSHAPERS FOR H/W TRAFFIC SHAPING



Net shapers is used to configure **hardware-based traffic shaping** (rate limiting) on network devices.



You can apply shaping rules at different levels, either on individual queues or on the entire network device.



Every shaper is uniquely identified using a handle, which includes a scope (like queue, device, or node) and an id.



queue scope: attached to a specific device queue
netdev scope: attached to the whole device
node scope: used to group other shapers and place them flexibly in the scheduling tree

PRIMARY NETSHAPER OPERATIONS



The **set** operation:
To create the shaper based on scope (queue or netdev)



The **get** operation:
To check the current shaping configuration.



The **delete** operation:
To remove or reset the shaper.

SENDING COMMANDS FROM USERSPACE

The shaping operations are performed using the **pyynl** CLI tool, which interfaces with the kernel via Netlink.

Run the following commands from the root of the latest Linux kernel source tree as **a root user**

Set Bandwidth Limit:

```
./tools/net/ynl/pyynl/cli.py \
   --spec Documentation/netlink/specs/net_shaper.yaml \
   --do set \
   --json '{"ifindex":3, "handle":{"scope": "netdev", "id":1 }, "bw-max": 200000000 }'
```

Get Shaper Details:

```
./tools/net/ynl/pyynl/cli.py \
   --spec Documentation/netlink/specs/net_shaper.yaml \
   --do get \
   --json '{"ifindex":3, "handle":{"scope": "netdev", "id":1 }}'
```

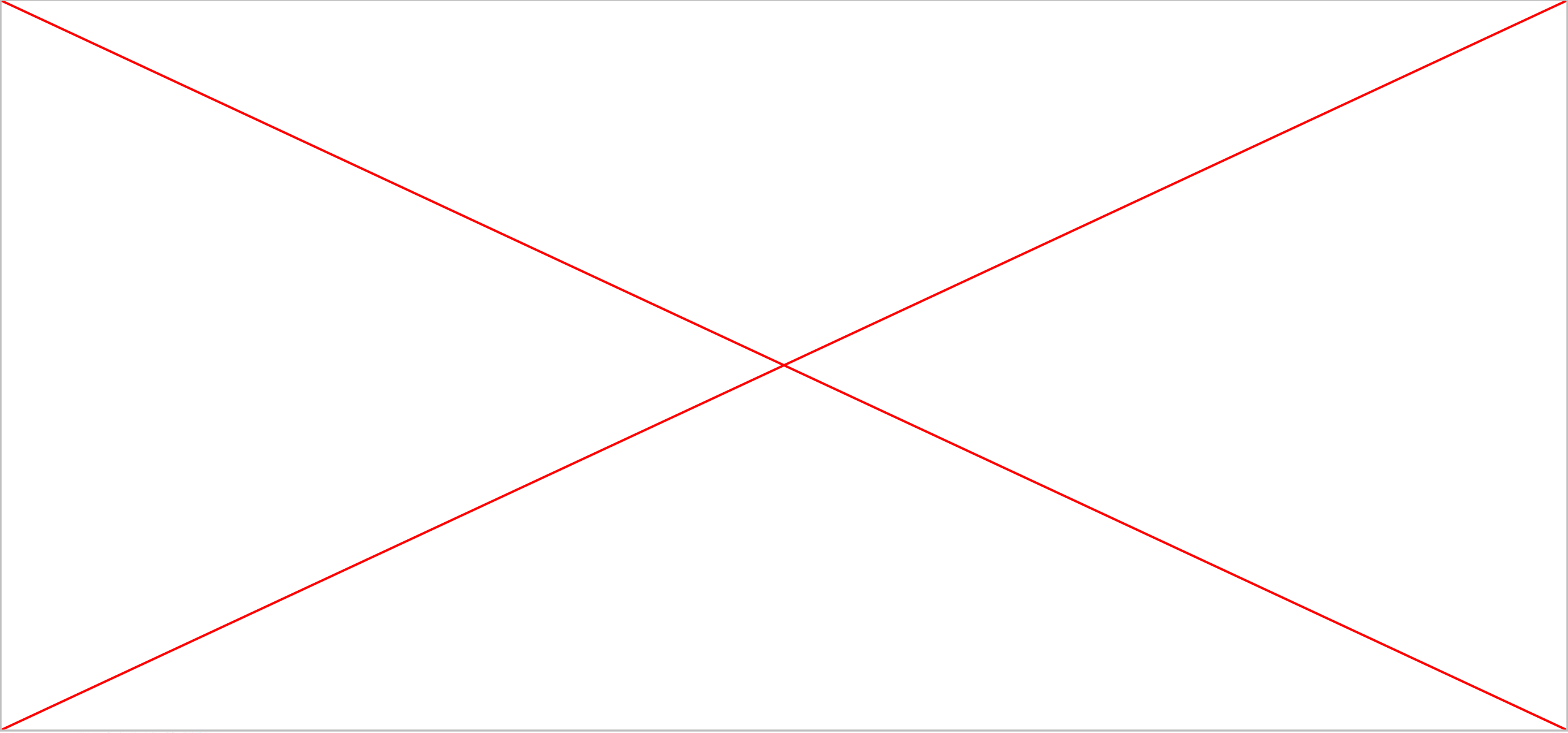
```
> {'bw-max': 200000000,
> 'handle': {'scope': 'netdev'},
> 'ifindex': $IFINDEX,
> 'metric': 'bps'}
```

Delete Shaper:

```
./tools/net/ynl/pyynl/cli.py \
   --spec Documentation/netlink/specs/net_shaper.yaml \
   --do delete \
   --json '{"ifindex":3, "handle":{"scope": "netdev", "id":1 }}'
```


NOTE:

- Make sure to enter the correct interface index (ifindex), since reloading for MANA module changes it's index. It can be obtained from [ip addr].
- The speed should be entered in bps (bits per second)



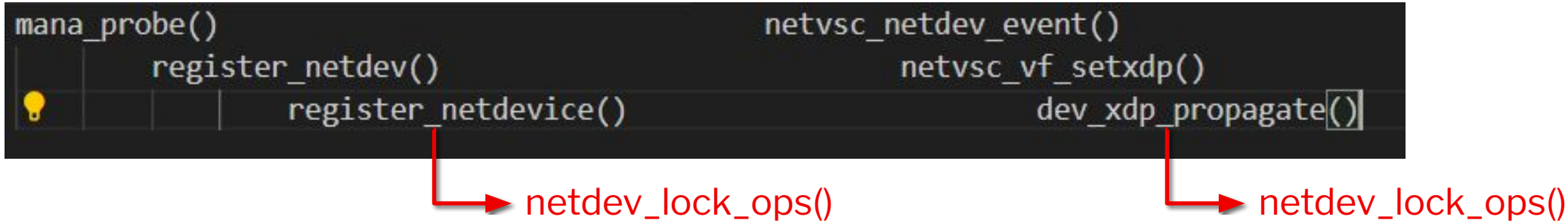

東京 2025
LINUX
PLUMBERS CONFERENCE
TOKYO, JAPAN / DEC. 11-13, 2025

CONTRAINTS

- Minimum Bandwidth: 100 Mbps
- Granularity: Must be a multiple of 100 Mbps
- H/W Setting: The Hardware setting is per VF i.e. in multi-vport scenario the latest speed setting will be used for all the vports.
- Scope: Only netdev scope is supported
- Single Shaper: Only one shaper per interface is allowed
- Error Recovery:
  On failure to set: previous speed and handle are restored
  On failure to delete: current configuration is retained

CHALLENGES FACED

- MANA and Netvsc drivers lacked proper netops support
- For eg,



- Similarly mana_open() and mana_close() face similar deadlock condition which were resolved on the fly.

CURRENT ISSUES WITH NETSHAPER IMPLEMENTATION IN MANA

1. There is no direct CLI to perform net-shaper operations without Linux src tree.
2. Users must specify the speed in bps, and if they want to use higher units like Gbps, they need to enter a long string of zeroes.
3. User must check the ifindex every time before using the commands.
4. The output of get operation is not very visually appealing.

SOLUTION

CURRENT ISSUES WITH NETSHAPER IMPLEMENTATION IN MANA

1. There is no direct CLI to perform net-shaper operations without Linux src tree.
2. Users must specify the speed in bps, and if they want to use higher units like Gbps, they need to enter a long string of zeroes.
3. User must check the ifindex every time before using the commands.
4. The output of get operation is not very visually appealing.

Introduce a new **iproute2 command** for net-shapers

Set Bandwidth Limit

YNL TOOL

```
./tools/net/ynl/pyynl/cli.py
--spec Documentation/netlink/specs/net_shaper.yaml
--do set
--json '{"ifindex":3, "handle":{"scope": "netdev",
"id":1 }, "bw-max": 200000000 }'
```

IPROUTE2 COMMAND

```
netshaper set dev <if> handle scope netdev
id 1 bw-max 200mbit
```

Delete Shaper

YNL TOOL

```
./tools/net/ynl/pyynl/cli.py
--spec Documentation/netlink/specs/net_shaper.yaml
--do delete
--json
'{"ifindex":3, "handle":{"scope": "netdev", "id":1 }}'
```

IPROUTE2 COMMAND

```
netshaper delete dev <if> handle scope
netdev id 1
```

Get Shaper Details

YNL TOOL

```
./tools/net/ynl/pyynl/cli.py
--spec Documentation/netlink/specs/net_shaper.yaml
--do get
--json '{"ifindex":3, "handle":{"scope": "netdev",
"id":1 }}'
```

```
{'bw-max': 200000000,
'handle': {'scope': 'netdev'},
'ifindex': $IFINDEX,
'metric': 'bps'}
```

IPROUTE2 COMMAND

```
netshaper show dev <if> handle scope netdev
id 1
```

```
scope : netdev
bw-max : 200 mbps
dev : <if>
```

FUTURE SCOPE

Current Status:
Userspace tool supports only the netdev scope and is limited to the MANA driver. Only one shaper per interface is allowed; minimum bandwidth is 100 Mbps, set in multiples of 100 Mbps.

```
enum net_shaper_scope {
        NET_SHAPER_SCOPE_UNSPEC,
        NET_SHAPER_SCOPE_NETDEV,
        NET_SHAPER_SCOPE_QUEUE,
        NET_SHAPER_SCOPE_NODE,

        /* private: */
        __NET_SHAPER_SCOPE_MAX,
        NET_SHAPER_SCOPE_MAX = (__NET_SHAPER_SCOPE_MAX - 1)
};
```

```
enum {
        NET_SHAPER_A_HANDLE = 1,
        NET_SHAPER_A_METRIC,
        NET_SHAPER_A_BW_MIN,
        NET_SHAPER_A_BW_MAX,
        NET_SHAPER_A_BURST,
        NET_SHAPER_A_PRIORITY,
        NET_SHAPER_A_WEIGHT,
        NET_SHAPER_A_IFINDEX,
        NET_SHAPER_A_PARENT,
        NET_SHAPER_A_LEAVES,

        __NET_SHAPER_A_MAX,
        NET_SHAPER_A_MAX = (__NET_SHAPER_A_MAX - 1)
};
```

Next Steps:
Expanding support to additional drivers and scopes (queue, node). Integrating more shaping features for broader hardware compatibility.

REFERENCES

1. Family net-shaper netlink specification — The Linux Kernel documentation
2. Netshaper Implementation in MANA
3. [PATCH net-next v3 0/4] Support bandwidth clamping in mana using net shapers - Erni Sri Satya Vennela
4. [PATCH net,v3] hv_netvsc: fix potential deadlock in netvsc_vf_setxdp() - Saurabh Sengar
5. [PATCH iproute2-next v4] netshaper: Add netshaper command - Erni Sri Satya Vennela

Thank you !!


東京 2025
LINUX
PLUMBERS CONFERENCE
TOKYO, JAPAN / DEC. 11-13, 2025



BACKUP SLIDES